Graphical Abstract

Multi-Agent Pickup and Delivery:
Formulations and Algorithms

Highlights

Multi-Agent Pickup and Delivery:
Formulations and Algorithms

- Research highlight 1
- Research highlight 2

Multi-Agent Pickup and Delivery: Formulations and Algorithms

[a], , , , ,

Abstract

Abstract text.

Keywords:

1. Introduction

Multi-Agent Pickup and Delivery (MAPD) [1] is a fundamental problem in multi-robot coordination, where a team of agents operates in a shared environment, continuously executing tasks that require visiting designated pickup and delivery locations while avoiding collisions. MAPD models a wide range of real-world applications, including automated warehouses, autonomous vehicle fleets, aircraft towing, and service robots, where tasks arrive over time and agents must interleave task assignment with collision-free path planning.

MAPD is challenging because it inherently couples two difficult problems: task assignment and Multi-Agent Path Finding (MAPF) [2]. Task assignment determines which agent executes which tasks and in what order, while path planning must account for dynamic interactions among agents in a shared space. Decisions made in one component directly affect the feasibility and quality of the other. As a result, MAPD algorithms must carefully balance effectiveness, computational efficiency, and robustness to deadlocks.

A substantial body of work has addressed MAPD and closely related problems. Early work focused on online MAPD [1], where tasks become known only at their release times, and proposed complete algorithms for a restricted but realistic subclass of instances, called well-formed instances. Subsequent work studied offline MAPD [3], where all tasks are known in advance, and demonstrated that assigning longer task sequences to agents can significantly improve performance. More recently, scalable variants [4] have been developed that trade completeness for efficiency by using windowed planning or limited-horizon replanning. In parallel, related research in MAPF [5], lifelong MAPF [6], and Multi-Robot Task Allocation (MRTA) [7] has explored different aspects of task sequencing, goal assignment, and collision avoidance.

Despite this progress, existing MAPD algorithms are often presented as standalone solutions tailored to specific settings (online vs. offline), planning horizons, or performance objectives. Key algorithmic mechanisms—such as instantaneous versus time-extended task assignment, single-goal versus multi-goal path planning, endpoint holding versus dummy-path reservation for deadlock avoidance, and event-driven versus periodic replanning—are typically introduced in an ad hoc manner and analyzed only within the context of a particular algorithm. As a result, it is difficult to systematically compare algorithms, transfer ideas across settings, or reason about correctness and scalability in a unified way.

The goal of this article is to provide a unified formulation and algorithmic framework for MAPD that clarifies these design choices and their theoretical implications. We distill a set of core mechanisms that underlie a broad class of existing approaches and show how different MAPD algorithms can be obtained by instantiating these mechanisms in different ways. This perspective allows us to (i) formally unify online, semi-online, and offline MAPD settings, (ii) cleanly separate task assignment, path planning, and deadlock avoidance, and (iii) reason about correctness and deadlock-freedom at the level of mechanisms rather than individual algorithms.

Building on this framework, we formalize MAPD and its multi-goal generalization (MG-MAPD), establish solvability conditions based on well-formedness, and revisit complexity results for common performance objectives. We then present a unified algorithmic framework that captures a wide range of MAPD algorithms, including those based on instantaneous or time-extended task assignment, sequential or windowed path planning, and different replanning triggers. A central focus of the paper is deadlock avoidance: We formalize endpoint holding and dummy-path reservation as two general mechanisms, prove their correctness under well-formedness, and show how they enable deadlock- and backtrack-free incremental replanning for arbitrary subsets of agents. We further position MAPD within the broader literature on MRTA and MAPF. In particular, we highlight how MAPD can be viewed as a form of MRTA with endogenous execution costs induced by multi-agent interactions, and how techniques from MAPF—such as multi-goal search and priority-based planning—can be systematically incorporated into MAPD solvers. Finally, we discuss extensions that integrate more accurate path-cost information into time-extended task assignment, including large-neighborhood search (LNS)-based methods, and evaluate representative algorithm instantiations on both MAPD and MG-MAPD benchmarks.

2. Multi-Agent Pickup and Delivery (MAPD)

In this section, we first formalize the problem of Multi-Agent Pickup and Delivery (MAPD) and then present theoretical results on its solvability and complexity. Finally, we discuss a multi-goal extension of MAPD.

2.1. Problem Definition

A MAPD problem instance consists of a set of M agents $\{a_1, a_2, ..., a_M\}$ and a connected undirected graph $G = (V, E)$, whose vertices V represent the set of locations and whose edges E represent the connections between locations that the agents can move along.

Paths. Let $\pi_i[t]$ denote the vertex occupied by agent a_i at time step t. Agent a_i starts at a unique initial vertex $\pi_i[0] = v_i^{\text{init}} \in V$. A *path* $\pi_i = \langle \pi_i[0], \pi_i[1], \ldots \rangle$ for agent a_i satisfies the following condition: At each time step, the agent always either moves to an adjacent vertex or waits at its current vertex, that is, for all time steps $t = 0, \ldots, \infty$, $(\pi_i[t], \pi_i[t+1]) \in E$ or $\pi_i[t+1] = \pi_i[t]$.

Collisions. Agents must avoid collisions with each other: A *vertex collision* is a tuple $\langle a_i, a_{i'}, v, t \rangle$ where agent a_i and agent $a_{i'}$ occupy the same vertex $v = \pi_i[t] = \pi_{i'}[t]$ at the same time step t. An *edge collision* is a tuple $\langle a_i, a_{i'}, u, v, t \rangle$ where agent a_i and agent $a_{i'}$ traverse the same edge (u, v), where $u = \pi_i[t] = \pi_{i'}[t+1]$ and $v = \pi_{i'}[t] = \pi_i[t+1]$, in opposite directions between time steps t and $t+1$.

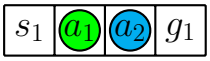

Figure 1: Example of an unsolvable MAPD instance on a 2D 4-neighbor grid.

Task Release, Assignment, and Execution. Each task τ_j is characterized by two *goals*—a *pickup vertex* $p_j \in V$ and a *delivery vertex* $d_j \in V$—and a finite *release time* $r_j \in \mathbb{N}$. A task can be assigned to one agent at a time. To execute task τ_j, an agent must first move from its current vertex to the pickup vertex p_j of the task and then move to the delivery vertex d_j of the task. When an agent arrives at p_i at or after time step r_i, it starts to execute τ_i and cannot execute other tasks. The *completion time* of τ_j is the time when the agent arrives at d_j. An agent is called *free* if and only if it is currently not executing any task. Otherwise, it is called *occupied*.

Online/Offline Settings and Look-Ahead Horizon. In the *online* setting, each task becomes known only at (and after) its release time, meaning that the *look-ahead horizon* is zero. In the *semi-online* setting, the system has a finite positive look-ahead horizon, and all tasks whose release times fall within that horizon are also known. In the *offline* setting, the look-ahead horizon is infinite, and thus all tasks are known at time step 0.

Effectiveness of MAPD Solving. The problem of MAPD is to assign tasks to agents and plan collision-free paths for them to complete all tasks. The following metrics are commonly used to evaluate the effectiveness of a MAPD algorithm:

1. Average service time (of all tasks): The *service time* of a task is the difference between its completion time and its release time (i.e., how long the task spends in the system).
2. Throughput (during a 100-step window): The throughput at time step t is the number of tasks completed in $[t-99, t]$.
3. Makespan: The *makespan* is the earliest time step when all tasks are completed.

The first two metrics are most relevant for the online and semi-online settings. Average service time reflects how quickly tasks are processed on average but does not indicate how well the system handles peak loads. Throughput captures dynamic performance—how many tasks can be processed concurrently—and sheds light on peak-load handling capacity; however, it can be low at the beginning or end of an episode when few tasks are available. Conversely, makespan is most relevant in the offline setting for batch completion; it is sensitive to outliers (a single long-running task can dominate) and may not reflect average performance in an online setting. In this article, we focus on these three metrics, while noting that other metrics—such as agent utilization, energy consumption, load balancing, and fairness—have also been considered in the literature.

2.2. Solvability and Well-Formedness

Not every MAPD instance is solvable. Below is an example.

Example 1. *Consider the MAPD instance shown in Figure 1 with two free agents a_1 and a_2. Neither agent can complete task τ_1 with pickup vertex p_1 and delivery vertex d_1.*

To ensure solvability, we introduce a sufficient condition called *well-formedness*. The intuition is that agents should only *terminally wait* (that is, remain in place without an intention to move away) at designated *endpoints*, where doing so does not block other agents. Endpoints are vertices at which agents may terminally wait; they include all initial vertices of agents and may

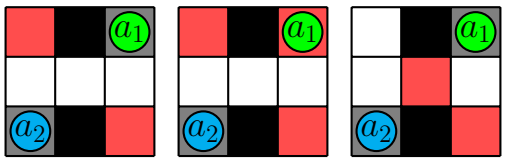

Figure 2: Three MAPD instances on 2D 4-neighbor grids. Blocked cells are in black. Task endpoints are in red. Non-task endpoints are in gray.

additionally include selected task goals and other designated vertices. Task goals that are not designated as endpoints must still be visited to execute tasks but are not vertices where agents are allowed to terminally wait. We define two types of endpoints: (1) all initial vertices of agents and (optionally) other designated endpoints are called *non-task endpoints*, and (2) optionally designated task goals are called *task endpoints*.

Definition 1. An MAPD problem instance is *well-formed* if and only if:

1. The number of tasks is finite.
2. The initial vertex of each agent is different from all task endpoints, and (thus) there are at least as many non-task endpoints as agents.
3. Between any two vertices in the set of endpoints and non-endpoint task goals, there exists a path whose intermediate vertices contain no endpoints.

Remark 1. *Well-formedness is defined to support incremental path finding in MAPD. By restricting terminal waiting to endpoints, path planning for an agent can always terminate at a vertex where the agent may safely wait indefinitely, which is essential when paths are planned and replanned online. The connectivity requirement ensures that, once other agents are terminally waiting at endpoints, any agent can still reach its assigned task goals without traversing occupied endpoints. The original MAPD formulation [1] designates all task goals as endpoints. This stronger assumption simplifies path finding by ensuring that every task completion location is also a valid terminal waiting location. Our definition generalizes this setting by allowing task goals that are not endpoints, provided that agents do not terminally wait at them, while preserving the key solvability and pathfinding guarantees.*

Example 2. *Figure 2 shows three MAPD problem instances. The left one is well-formed. The center one is not, because it has two agents but only one non-task endpoint. The right one is not, because every path between its two non-task endpoints passes through the central task endpoint.*

Theorem 1. *All well-formed MAPD problem instances are solvable.*

Proof. Consider the following trivial algorithm for a well-formed instance: Assign any single agent to execute all tasks in any order, while all other agents terminally wait at their initial vertices (which are non-task endpoints). By well-formedness, this agent can complete all tasks without having to traverse the initial vertices of other agents. □

2.3. Complexity

We first show a constant-factor inapproximability result for MAPD for makespan minimization via a reduction from the NP-complete ≤3,=3-SAT problem [8]. A ≤3,=3-SAT instance

consists of $\mathcal{N}$ Boolean variables and $\mathcal{M}$ disjunctive clauses where each variable appears in exactly three clauses, uncomplemented at least once and complemented at least once, and each clause contains at most three literals. The decision question asks whether there exists a satisfying assignment.

Theorem 2. *For any $\epsilon > 0$, it is NP-hard to find a $(4/3 - \epsilon)$-approximate solution to MAPD for minimizing the makespan.*

Proof Sketch. Our construction closely follows the proof of Theorem 3 in [9], adapted to MAPD. We use the same notations and point out the differences in the construction: We create a MAPD instance with $M = 2\mathcal{N} + \mathcal{M}$ agents and the same number of tasks. Each task has a release time 0, and its pickup vertex initially matches the initial vertex of a designated agent. Specifically, for each variable X_i, we construct two "literal" agents a_{iT} and a_{iF} with initial vertices s_{iT} and s_{iF}, respectively, and two tasks τ_{iT} and τ_{iF} with pickup vertices s_{iT} and s_{iF} and delivery vertices t_{iT} and t_{iF}, respectively. For each clause C_j, we construct a "clause" agent a_j with initial vertex c_j and a task τ_j with pickup vertex c_j and delivery vertex d_j. Therefore, any optimal solution must assign each task to the agent whose initial vertex is the pickup vertex of the task at time step 0 and let the agent execute the task.
To summarize, using the same arguments as in the proof of Theorem 3 in [9], the MAPD instance has a solution with makespan 3 if and only if the ≤3,=3-SAT instance is satisfiable. Also, the MAPD instance cannot have a solution with makespan smaller than 3 and always has a solution with makespan 4, even if the ≤3,=3-SAT instance is unsatisfiable. For any $\epsilon > 0$, any approximation algorithm for MAPD with ratio $4/3 - \epsilon$ thus computes a solution with makespan 3 whenever the ≤3,=3-SAT instance is satisfiable, effectively solving ≤3,=3-SAT. □

We note that, in the constructed MAPD instance, each task requires at least three time steps to finish. Therefore, if the makespan is three, every task is completed in exactly three time steps, and the average service time is three. Moreover, if the makespan exceeds three, the average service time also exceeds three, yielding the following corollary:

Corollary 3. *It is NP-hard to find an optimal solution to MAPD for minimizing the average service time.*

2.4. Extension: Multi-Goal MAPD

Multi-Goal MAPD (MG-MAPD) generalizes MAPD by allowing each task τ_j to have an ordered sequence of goals, with the first goal being the pickup vertex p_j and the last goal being the delivery vertex d_j. To execute τ_j, an agent must visit all goal locations of τ_j in the specified order. MAPD is thus a special case of MG-MAPD where each sequence contains exactly two goals (pickup and delivery). The theoretical analysis presented above and most of the MAPD algorithms apply to MG-MAPD without special modification. Therefore, the technical content in this article focuses primarily on MAPD, while we additionally present experimental results on MG-MAPD to demonstrate the generality of our approach.

3. Related Work

In this section, we discuss techniques for the two core components of MAPD: task assignment and path finding. We then survey existing combined task-assignment and path-finding problem formulations.

3.1. Task Assignment

Task assignment in MAPD is closely related to the multi-robot task allocation literature [10, 11]. In the taxonomy of [10], MAPD falls under the categories of *single-task robots* (each agent can execute at most one task at a time) and *single-robot tasks* (each task requires exactly one agent). We discuss two relevant types of task-assignment approaches:

Instantaneous Assignment (IA). IA is best suited to settings where real-time constraints limit both the available information about future tasks and the computation time. IA attempts to assign each agent at most one task without planning for future task releases. Formally, it solves a bipartite matching problem for m agents and n tasks to minimize the sum of assignment costs, which typically capture the travel time for an agent to reach the pickup vertex of a task or to complete the entire task in MAPD. The Hungarian method [12] computes an optimal matching in $O(mn^2)$ time.

Time-Extended Assignment (TA). TA is more applicable when the system may know of tasks well in advance and can devote more computation time. TA attempts to assign all tasks to agents in a way that minimizes the total cost. Since the cost depends on the order in which each agent executes its assigned tasks in MAPD, TA can be viewed as solving variants of multiple Traveling Salesman Problem (mTSP) or Vehicle Routing Problem (VRP) [13, 14], both of which are strongly NP-hard. Large Neighborhood Search [14] is a popular local-search strategy for VRPs: starting from an initial solution, it iteratively removes a subset of tasks and re-inserts them using a greedy heuristic, thus continually improving the solution.

3.2. Path Finding

Path finding in MAPD is closely related to MAPF [2], which aims to compute collision-free paths for all agents to their respective (preassigned) goals. Minimizing the sum of path costs or the makespan for MAPF is NP-hard in general [15, 16, 9]. Many MAPF algorithms have been proposed, including the complete and optimal Conflict-Based Search (CBS) [17] (and its improved variants [18, 19, 20]) as well as the incomplete and suboptimal Prioritized Planning (PP) [21]. PP assigns a fixed total ordering to agents and then computes time-minimal paths one agent at a time, ensuring that each new path does not collide with previously computed (higher-priority) paths. Priority-Based Search (PBS) [22] extends PP by systematically searching for an effective priority ordering rather than fixing one.

Several relevant MAPF variants have also been studied. Lifelong MAPF [6] continuously assigns new goals to each agent once it arrives at its current goal, following a predefined goal sequence. Online MAPF [23, 24] accommodates newly revealed agents by solving a one-shot MAPF instance whenever a new agent appears. Goal-Sequencing MAPF [25, 26] requires computing a good order in which each agent should visit its predefined set of goals.

3.3. Combined Task Assignment and Path Finding

Other problem formulations combine task assignment and MAPF, paralleling MAPD. One prominent example is Combined Target Assignment and Path Finding (TAPF) [5, 27], which seeks a one-to-one assignment of goals to agents and collision-free paths that jointly minimize the sum of path costs or the makespan. Several generalizations [28, 29, 30] extend TAPF to allow multiple goals per agent, as in MAPD, although they typically address only the offline setting. An online version of TAPF [31] has been studied, focusing on reducing agent idle time. However, none of these TAPF variants supports pickup-and-delivery operations.

Algorithm 1: Unified MAPD Framework

Input : MAPD instance, look-ahead horizon L

1 $t \leftarrow 0$;
2 $\mathcal{T} \leftarrow \emptyset$;
3 **if** OFFLINE **then**
4 $\quad \mathcal{T} \leftarrow$ all tasks;
5 Initialize task sequences for all agents;
6 Initialize each π_i with the trivial path $\pi_i[0] = v_i^{\text{init}}$;
7 **while** $\neg$ END() **do**
8 $\quad$ **if** ONLINE **then**
9 $\quad\quad \mathcal{T} \leftarrow \mathcal{T} \cup \{\tau_j \mid r_j = t\}$;
10 $\quad$ **else if** SEMI-ONLINE **then**
11 $\quad\quad \mathcal{T} \leftarrow \mathcal{T} \cup \{\tau_j \mid r_j \leq t + L\}$;
12 $\quad$ COMPUTATION: (Re)assign tasks in $\mathcal{T}$ to agents and (re)plan paths, updating task sequences, paths, and $\mathcal{T}$ accordingly;
13 $\quad$ Each agent moves one step along its path, $t \leftarrow t + 1$;

4. Unified Algorithmic Framework for MAPD

In this section, we present an algorithmic framework that iteratively assigns tasks and plans paths for agents, unifying a wide range of MAPD algorithms across online, semi-online, and offline settings. A fundamental challenge in MAPD is that task assignment and collision-free path finding are tightly coupled: Solving them jointly as a single optimization problem is computationally expensive and quickly becomes impractical even for small offline instances [29]. In contrast, in online and semi-online settings, tasks are revealed over time, which naturally favors incremental decision making rather than a one-shot, fully time-extended formulation.

Consequently, most practical MAPD algorithms follow an iterative structure, which is a natural fit for the online setting, where tasks are revealed over time and assignment decisions are necessarily myopic, often taking the form of instantaneous assignment inspired by MRTA. Even in offline settings, iterative methods provide a practical alternative to fully coupled approaches by repeatedly updating task assignments and paths as tasks are completed. However, unlike classical MRTA, MAPD requires that every intermediate task assignment admit a feasible set of collision-free paths. Without additional structure, naive combinations of task assignment and sequential path planning can easily render future replanning infeasible or lead to deadlocks. Our framework formalizes this iterative approach and identifies the conditions and algorithmic mechanisms required to safely integrate task assignment and path planning.

4.1. Pseudo-Code

Algorithm 1 outlines the pseudo-code of our algorithmic framework. The time starts at zero [Line 1], and the *task set* $\mathcal{T}$, which stores all available unexecuted tasks, is initially empty [Line 2]. In the offline setting, $\mathcal{T}$ is immediately populated with all tasks [Line 4]. The task sequence of each agent (the ordered list of tasks assigned to it) is initialized [Line 5], and the path of each agent starts with its initial vertex [Line 6]. Our framework assumes that paths are always planned to endpoints, so each path π_i always ends at an endpoint and the paths are collision-free throughout the execution of our framework.

The framework proceeds in time steps until End() returns true [Lines 7]. In an offline setting, End() may return true once all tasks have been completed; in a continuously operating system, it may always return false. At each time step t: In the online setting, all tasks with release time t are added to $\mathcal{T}$ [Line 9]; in the semi-online setting, all tasks with release time up to $t+L$ are added to $\mathcal{T}$ [Line 11]. Next, the core computation [Line 12] assigns tasks from $\mathcal{T}$ to agents (as necessary) and plans their paths. Then, each agent moves one step along its path, and time advances to $t+1$ [Line 13].

Although the pseudo-code shows task assignment and path planning happening every time step, many MAPD algorithms perform these computations less frequently—for example, only when an agent completes a task (thus becoming free), when a new task is added to $\mathcal{T}$, or at a fixed frequency (e.g., every few time steps). Section 4.6 summarizes different design choices for when and how to perform these computations. In the following, we present the key components of our framework.

4.2. Time-Optimal Single-Agent Path Finding

The core computation of our framework involves planning an effective path for each agent from its current vertex to a goal or through a sequence of goals, while avoiding collisions with the paths of other agents. We discuss two A* search approaches for time-optimal single-agent path finding. Since all goals in MAPD are endpoints, the shortest-path distance $\mathrm{dist}(v, g)$ from all vertices v to each endpoint g can be precomputed to obtain an admissible heuristic.

Space-Time A (STA*).* STA*[32] plans a path for a single agent from its current vertex to a single goal $g \in V$. It is an A* search whose states are pairs $\langle v, t\rangle$ of a vertex v and a time step t. Its state space has a directed edge from $\langle u, t\rangle$ to $\langle v, t+1\rangle$ if and only if $u = v \in V$ or $(u, v) \in E$. STA* ensures collision avoidance by removing any constrained states (i.e., $\langle v, t\rangle$ that the agent is constrained not to occupy) and edges (i.e., $\langle u, t\rangle \to \langle v, t+1\rangle$ that the agent is constrained not to traverse). This yields a *time-minimal path* (i.e., with the minimum arrival time at the goal). STA* uses the precomputed $\mathrm{dist}(v, g)$ as the admissible heuristic value for each state $\langle v, t\rangle$.

Multi-Label (Space-Time) A (MLA*).* MLA* [33, 6] generalizes STA* to handle an ordered sequence of goals $\boldsymbol{g}[1], \ldots, \boldsymbol{g}[K]$. Its search states are tuples $\langle v, t, k\rangle$ of a vertex v, a time step t, and a label k indicating that the next goal to visit is $\boldsymbol{g}[k]$. Its state space has a directed edge from state $\langle u, t, k\rangle$ to state $\langle v, t+1, k'\rangle$ if and only if (1) $u = v$ or $(u, v) \in E$ and (2) $k' = k+1$ if $v = \boldsymbol{g}[k]$ and $k' = k$ otherwise. Collision avoidance is enforced in the same way as for STA* by removing forbidden states and edges. MLA* uses $\mathrm{dist}(v, \boldsymbol{g}[k]) + \sum_{k'=k}^{K-1} \mathrm{dist}(\boldsymbol{g}[k'], \boldsymbol{g}[k'+1])$ as the admissible heuristic value for each state $\langle v, t, k\rangle$, which represents the shortest path distance from v through all remaining goals in the sequence. STA* is a special case of MLA* with $k = 1$.

4.3. Incompleteness of Sequential A Calls*

Solving MAPD within our framework often involves planning a collision-free path for a single agent through a sequence of goals (e.g., pickup followed by delivery), while new tasks may arrive over time. Although MLA* can find a time-minimal path for a fully known goal sequence, in practice, MAPD algorithms commonly rely on multiple, sequential calls to (ST or ML)A*—each time planning only up to the next goal (or the next few goals) and then concatenating the resulting paths. This approach arises either because the task-assignment module assigns an agent only the next (few) task(s) at a time—thus the full goal sequence is not known—or because it is more computationally efficient.

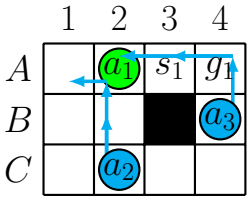

Figure 3: Planning a collision-free path on a 2D 4-neighbor grid.

We now show that this sequential approach, without additional safeguards, can be incomplete (and thus suboptimal). Example 3 demonstrates how following the path returned by an initial A* call and then planning from the last state of that path can cause deadlocks in the subsequent call, which motivates the deadlock-avoidance mechanisms in our framework.

Example 3. *Figure 3 shows a 2D 4-neighbor grid where agent a_1 must visit $p_1 = A3$ and then $d_1 = A4$, while avoiding collisions with the paths $\pi_2 = C2 \to B2 \to A2 \to A1$ and $\pi_3 = B4 \to A4 \to A3 \to A2$ of agents a_2 and a_3. A naive sequential STA* approach first finds the path $A2 \to A3$. However, the subsequent search from $\langle A3, 1\rangle$ fails to find any valid path to $d_1 = A4$ (in fact, a_1 is immediately deemed to collide with a_2 or a_3) because it cannot backtrack to arrive at $A3$ at a different time step. A solution does exist if a_1 takes a longer initial path to $A3$, for example, $A2 \to A1 \to B1 \to C1 \to C2 \to C3 \to C4 \to B4 \to A4 \to A3 \to A4$.*

4.4. Endpoint Holding and Deadlock Avoidance

To address the incompleteness of sequential path planning illustrated in Example 3, we introduce a unifying abstraction based on the notion of a last endpoint of a planned path. This abstraction subsumes several deadlock-avoidance techniques used in prior MAPD algorithms.

4.4.1. Last Endpoint Holding

In our framework, every planned path π_i for agent a_i is required to end at a designated endpoint $\ell_i \in V$, called the *last endpoint* of the path. Agent a_i is assumed to *terminally wait* at ℓ_i after reaching it, meaning that ℓ_i is reserved by a_i from its arrival time onward and must be avoided by all other agents. We refer to this invariant as *Endpoint Holding*.

Initially, each agent a_i has the trivial path $\pi_i[0] = v_i^{\text{init}}$ (Line 6 of Algorithm 1), whose last endpoint is its initial vertex. Regardless of how paths are updated over time, Endpoint Holding maintains the invariant that every agent can follow its current path and, if necessary, wait indefinitely at its last endpoint. The following result formalizes the key implication of Endpoint Holding under well-formedness (Definition 1).

Theorem 4. *Consider a well-formed MAPD instance using Endpoint Holding, where agents currently follow collision-free paths $\{\pi_i\}$ with last endpoints $\{\ell_i\}$. For any agent a_i with its current path π_i, there exists a collision-free path from any state (vertex $\pi_i[t]$ at time step t) along π_i to visit an ordered sequence of endpoints $\boldsymbol{g}[1], \ldots, \boldsymbol{g}[K]$ if no other agent holds any of them.*

Proof. Since the current $\{\pi_i\}$ are collision-free, a_i can complete π_i and wait at its last endpoint ℓ_i, while all other agents complete their paths and terminally wait at theirs. By well-formedness, there exists a path from ℓ_i to $\boldsymbol{g}[1]$ that avoids all other endpoints, and similarly between successive endpoints in the sequence. Therefore, a_i can visit $\boldsymbol{g}[1], \ldots, \boldsymbol{g}[K]$ in order without traversing any endpoint occupied by another agent and terminally wait at $\boldsymbol{g}[K]$. □

This theorem provides a general correctness guarantee for incremental path replanning: Once a path is planned to a suitable last endpoint, future pathfinding problems remain feasible as long as no two agents hold the same endpoint.

4.4.2. Last Endpoint Choices, Dummy Path, and Deadlock Avoidance

Building on Theorem 4, we view deadlock avoidance in well-formed MAPD instances as enforcing endpoint holding with an explicit choice of the last endpoint ℓ_i of each planned path π_i.

In many MAPD algorithms, sequential (ST or ML)A* calls are intended to reach a sequence of task-related endpoints $\boldsymbol{g}[1], \ldots, \boldsymbol{g}[K]$ (e.g., pickup or delivery vertices), while the desired last endpoint ℓ_i may or may not coincide with $\boldsymbol{g}[K]$. When $\ell_i \neq \boldsymbol{g}[K]$, the planner must additionally ensure the existence of a collision-free continuation (a suffix) from $\boldsymbol{g}[K]$ to ℓ_i. We refer to this continuation as a *dummy path*. The dummy path is not a separate planning objective; rather, it serves as a feasibility condition that certifies that reaching $\boldsymbol{g}[K]$ does not preclude future replanning under endpoint holding.

The dummy-path condition can be enforced in two standard ways:

1. **Goal-test verification.** Modify the goal test of the main (ST or ML)A* search so that $\langle \boldsymbol{g}[K], t \rangle$ is accepted as a goal state only if an additional STA* search from $\langle \boldsymbol{g}[K], t \rangle$ can find a collision-free path to ℓ_i, whose goal test enforces terminal waiting at ℓ_i. If no such path exists, the main search continues.
2. **Explicit goal augmentation.** Append ℓ_i to the goal sequence and use MLA* to plan a single path that visits $\boldsymbol{g}[1], \ldots, \boldsymbol{g}[K]$ and then ℓ_i, enforcing endpoint holding at ℓ_i in the final goal test.

Both implementations are complete whenever a feasible dummy path exists. Implementation (1) preserves time optimality for reaching $\boldsymbol{g}[K]$, while implementation (2) minimizes arrival at ℓ_i and may delay reaching $\boldsymbol{g}[K]$. In practice, this can be mitigated by using a lexicographic cost function in MLA* that first minimizes the arrival time at $\boldsymbol{g}[K]$ and then the arrival time at ℓ_i.

We highlight three representative choices adopted by existing MAPD algorithms:

1. **Current task endpoint.** Set $\ell_i = \boldsymbol{g}[K]$, which corresponds to "holding a task endpoint" adopted in [1] and can be viewed as a zero-length dummy path.
2. **Fixed non-task endpoint.** Fix ℓ_i to be a unique non-task endpoint for each agent (e.g., its initial vertex), adopted in [3].
3. **Flexible last endpoint.** Choose ℓ_i flexibly from a set of admissible endpoints, typically all non-task endpoints or all endpoints (including task endpoints) that are not delivery vertices of available unexecuted tasks (adopted in [4]).

Remark 2. *Since the dummy path exists solely to guarantee that the subsequent search (which plans a path to replace this dummy path) remains feasible, an agent rarely executes it—except when no tasks remain. Different choices of last endpoints differ in performance trade-offs but all enforce the same endpoint-holding invariant. Choosing ℓ_i to be any task endpoint can block other agents from executing tasks that share the same endpoint, while choosing ℓ_i to be a non-task endpoint avoids such blocking at the cost of additional movements. A practical advantage of planning to a single goal and choosing ℓ_i to be the current task endpoint arises when an agent must remain at that endpoint for an unknown duration (e.g., load/unload operations), in which case endpoint holding naturally captures the earliest feasible departure time for subsequent planning.*

4.5. Deadlock- and Backtrack-Free Subset Replanning

We now show how Endpoint Holding, combined with appropriate choices of last endpoints, enables deadlock- and backtrack-free replanning for an arbitrary subset $\mathcal{A}$ of agents.

Assumption 1. Consider a well-formed MAPD instance using Endpoint Holding, where agents currently follow their collision-free paths $\{\pi_i\}$ with last endpoints ℓ_i. Let $\mathcal{A}$ be any subset of agents. For each agent $a_i \in \mathcal{A}$, assume that a new ordered sequence of endpoints $\boldsymbol{g}_i[1], \ldots, \boldsymbol{g}_i[K_i]$ is assigned.

Theorem 5. *Under Assumption 1, assume also that any $\boldsymbol{g}_i[k]$ assigned to each a_i is (1) distinct from $\boldsymbol{g}_{i'}[K_{i'}]$ assigned to any other $a_{i'} \in \mathcal{A}$ and (2) not the current last endpoint of any agent outside $\mathcal{A}$. Then there exist collision-free paths for all agents in $\mathcal{A}$ from their current states (i.e., some $\pi_i[t]$ at time step t) to visit their assigned endpoint sequences in order and terminally wait at $\boldsymbol{g}_i[K_i]$.*

Proof. Since the current paths π_i are collision-free, all agents can first complete their paths and terminally wait at their respective last endpoints. By condition (2), no agent outside $\mathcal{A}$ holds any endpoint in the assigned sequences. Similar to the proof of Theorem 4, the agents in $\mathcal{A}$ can move one after another to visit their assigned endpoint sequences, with one exception: If, at some step, an endpoint $\boldsymbol{g}_i[k]$ is temporarily occupied, it can only be occupied by another $a_{i'} \in \mathcal{A}$ that has not yet moved to the new last endpoint $\boldsymbol{g}_{i'}[K_{i'}]$ (e.g., it is the current last endpoint of $a_{i'}$). In this case, $a_{i'}$ moves to a different unoccupied endpoint, whose existence is guaranteed by well-formedness. Meanwhile, by conditions (1) and (2), the agents in $\mathcal{A}$ that have moved to their new last endpoints and all agents outside $\mathcal{A}$ remain unmoved and never block $\boldsymbol{g}_i[k]$. □

The following theorem shows that, by additionally avoiding assigning the current last endpoints of agents in $\mathcal{A}$ in condition (2) of Theorem 5, the paths for agents in $\mathcal{A}$ can be planned one at a time in any order, i.e., using PP.

Theorem 6. *Under Assumption 1, assume also that any $\boldsymbol{g}_i[k]$ assigned to each a_i is (1) distinct from $\boldsymbol{g}_{i'}[K_{i'}]$ assigned to any other $a_{i'} \in \mathcal{A}$ and (2) not the current last endpoint of any agent. Then there exist collision-free paths for all agents in $\mathcal{A}$ from their current states (some $\pi_i[t]$ at time step t) to visit their assigned endpoint sequences in order. Moreover, these paths can be planned for the agents in $\mathcal{A}$ one at a time in any order: For each agent $a_i \in \mathcal{A}$, there exists such a path that does not collide with the updated paths of agents already replanned in $\mathcal{A}$ and the current paths of all remaining agents in $\mathcal{A}$ and agents outside $\mathcal{A}$.*

Proof. Since the current paths $\{\pi_i\}$ are collision-free, all agents can first complete their paths and terminally wait at their respective last endpoints. By condition (2), no other agent (inside or outside $\mathcal{A}$) holds any endpoint in the assigned sequences. Similar to the proof of Theorem 5, the agents in $\mathcal{A}$ can move one after another to visit their assigned endpoint sequences, while avoiding collisions with the updated paths of agents already replanned in $\mathcal{A}$ due to condition (1) and the current paths of all remaining agents in $\mathcal{A}$ and agents outside $\mathcal{A}$ due to condition (2). □

In particular, assigning each agent a distinct non-task endpoint as its last endpoint is sufficient to satisfy the assumptions of Theorem 6.

4.6. Task Assignment

We follow the classification from Section 3.1: Instantaneous Assignment (IA) assigns at most one unexecuted task to each free agent at a time, while Time-Extended Assignment (TA) assigns all unexecuted tasks at once, computing a task sequence to each agent. Although MAPD naturally suggests a TA formulation, IA offers a simpler but more myopic alternative by iteratively assigning tasks in smaller batches rather than performing a global, time-extended optimization.

4.6.1. IA Methods

IA methods typically define the cost of assigning a task τ_j to an agent a_i as the path cost from the current vertex of a_i to the pickup vertex p_j. They then assign tasks to minimize the sum of these assignment costs. In this article, we consider three common methods:

1. **Decoupled Greedy** considers one agent at a time and assigns each agent the task with the smallest cost.
2. **Centralized Greedy** selects the task-agent pair with the smallest cost and then repeats until no tasks or agents remain.
3. **Optimal Matching** uses the Hungarian method [12] to compute a minimum-cost matching.

For efficiency, IA methods typically use the precomputed shortest-path distance $\mathrm{dist}(v, p_j)$ from the current vertex v of a_i to p_j as a lower bound on the true collision-free path cost. Alternatively, they can be coupled with full path planning to obtain exact collision-free path costs.

Remark 3. *An alternative cost definition is the path cost to complete τ_j—from the current vertex to visit all goals of τ_j. This cost directly aligns with the service time but may over-prioritize tasks with shorter tours, potentially starving tasks with distant pickup and delivery vertices in practice.*

In the context of Algorithm 1, an IA-based MAPD method simply repeatedly solves an IA instance on Line 12 until all unexecuted tasks are assigned and completed.

4.6.2. TA Methods

TA is most relevant in offline or semi-online settings, assigning all or a batch of unexecuted tasks at once. TA yields one *task sequence* for each agent a_i, specifying which tasks the agent executes and in what order. Even when not coupled with collision-free path planning, TA is computationally challenging, as it resembles an mTSP variant with two complications:

- Asymmetric Distances: The cost of assigning τ_j followed by $\tau_{j'}$ depends on the distance from d_j to $p_{j'}$, whereas reversing the order depends on the distance from $d_{j'}$ to p_j.
- Dynamic Distances: Tasks have release times, so an agent a_i may wait before starting a task. The resulting *execution time* $\mathcal{M}_i$ (the total time steps for a_i to complete its assigned tasks) can differ from the travel distance.

In a semi-online setting, TA typically optimizes the average service time of all tasks. In an offline setting, TA typically optimizes the makespan, defined as the maximum $\mathcal{M}_i$ among all agents. In this article, we use both an off-the-shelf TSP solver and a specialized LNS method to tackle TA.

4.7. Integrating Task Assignment and Path Finding

Table 1 summarizes representative MAPD algorithms that fit into our unified framework, each reflecting different design choices regarding: Which tasks to consider for assignment, which agents to assign, when and for which agents to plan paths, whether task assignment must precede every path-planning step, and what methods to use for task assignment and path planning. The table also highlights key properties such as collision-avoidance mechanisms and completeness for well-formed instances.

		task assignment				path planning			collision avoidance	complete for well-formed instances
original setting & paper	algorithm	type	when to	method	coupled w/ path planning?	when & which agents to plan	MAPF method	single-agent method		
online [1]	TP	IA	when any free agent terminally waits	Decoupled Greedy	no	after each task assignment: all free agents, one at a time	decoupled	sequential STA* calls to execute a task or STA* to a non-task endpoint	Holding Endpoint	yes
	TPTS	IA	when any free agent terminally waits	Decoupled Greedy w/ swaps	swaps only	after each task assignment: all free agents, one at a time	decoupled	sequential STA* calls to execute a task or STA* to a non-task endpoint	Holding Endpoint	yes
	CENTRAL	IA	every time step	Hungarian	no	every time step: [Group1] all agents becoming occupied & [Group2] all free agents	CBS	STA* to the next endpoint	Holding Endpoint	no
online [34]	CENTRAL (fixed)	IA	when any new task arrives or when any agent becomes free	Hungarian	no	[Group1] when any agent becomes occupied: all such agents & [Group2] after each task assignment: all free agents	CBS	STA* to the next endpoint	Holding Endpoint	yes
online [33]	HBH-MLA*	IA	when any free agent terminally waits	Centralized Greedy	no	after each task assignment: one free agent at a time	decoupled	MLA* to execute a task or to a non-task endpoint	Holding Endpoint	yes
offline [3]	TA-Prioritized	TA	once	LKH3 TSP	no	once: all agents	PP	sequential STA* calls to execute all tasks in the sequence	Dummy Path	yes
	TA-Hybrid	TA	once & when any agent becomes free (reassign)	LKH3 TSP w/ reassign	reassign only	[Group1] when any agent becomes occupied: all such agents & [Group2] when any agent becomes free: all free agents	[Group1] ICBS & [Group2] min-cost max flow	STA* to the next endpoint	Dummy Path	yes
offline & online [35]	RMCA	TA	once (offline) & when any new task arrives (online)	greedy insertion + LNS	yes	after each task assignment: all agents, one at a time, coupled with task assignment	decoupled	STA* to the next endpoint	none	no
online & offline & semi-online [4]	LNS-PBS	TA	when any task remains unassigned or when any agent becomes free	repeated Hungarian + LNS	no	after each task assignment: all agents	PBS	MLA* to execute all tasks in the sequence	Dummy Path	yes
	LNS-wPBS	TA	when any task remains unassigned or when any agent becomes free	repeated Hungarian + LNS	no	after each task assignment & every w time steps: all agents	wPBS	MLA* to execute all tasks in the sequence	Dummy Path	no

Table 1: Overview of different MAPD algorithms within our unified framework.

References

[1] H. Ma, J. Li, T. K. S. Kumar, S. Koenig, Lifelong multi-agent path finding for online pickup and delivery tasks, in: International Conference on Autonomous Agents and Multiagent Systems, 2017, pp. 837–845.

[2] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski, R. Bartak, Multi-agent pathfinding: Definitions, variants, and benchmarks, in: International Symposium on Combinatorial Search, 2019, pp. 151–159.

[3] M. Liu, H. Ma, J. Li, S. Koenig, Task and path planning for multi-agent pickup and delivery, in: International Conference on Autonomous Agents and Multiagent Systems, 2019, pp. 2253–2255.

[4] Q. Xu, J. Li, S. Koenig, H. Ma, Multi-goal multi-agent pickup and delivery, in: IEEE/RSJ International Conference on Intelligent Robots and System, 2022, pp. 9964–9971.

[5] H. Ma, S. Koenig, Optimal target assignment and path finding for teams of agents, in: International Conference on Autonomous Agents and Multiagent Systems, 2016, pp. 1144–1152.

[6] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, S. Koenig, Lifelong multi-agent path finding in large-scale warehouses, in: AAAI Conference on Artificial Intelligence, 2021, pp. 11272–11281.

[7] G. A. Korsah, A. Stentz, M. B. Dias, A comprehensive taxonomy for multi-robot task allocation, International Journal of Robotics Research (2013) 1495–1512.

[8] C. Tovey, A simplified NP-complete satisfiability problem, Discrete Applied Mathematics 8 (1984) 85–90.

[9] H. Ma, C. Tovey, G. Sharon, T. K. S. Kumar, S. Koenig, Multi-agent path finding with payload transfers and the package-exchange robot-routing problem, in: AAAI Conference on Artificial Intelligence, 2016, pp. 3166–3173.

[10] B. P. Gerkey, M. J. Matarić, A formal analysis and taxonomy of task allocation in multi-robot systems, The International Journal of Robotics Research 23 (2004) 939–954.

[11] G. A. Korsah, A. Stentz, M. B. Dias, A comprehensive taxonomy for multi-robot task allocation, The International Journal of Robotics Research 32 (2013) 1495–1512.

[12] H. W. Kuhn, The Hungarian method for the assignment problem, Naval Research Logistics Quarterly 2 (1955) 83–97.

[13] T. Bektas, The multiple traveling salesman problem: an overview of formulations and solution procedures, Omega 34 (2006) 209–219.

[14] P. Shaw, A new local search algorithm providing high quality solutions to vehicle routing problems, APES Group, Dept of Computer Science, University of Strathclyde 46 (1997).

[15] P. Surynek, An optimization variant of multi-robot path planning is intractable, in: AAAI Conference on Artificial Intelligence, 2010, pp. 1261–1263.

[16] J. Yu, S. M. LaValle, Structure and intractability of optimal multi-robot path planning on graphs, in: AAAI Conference on Artificial Intelligence, 2013, pp. 1444–1449.

[17] G. Sharon, R. Stern, A. Felner, N. R. Sturtevant, Conflict-based search for optimal multi-agent pathfinding, Artificial Intelligence 219 (2015) 40–66.

[18] J. Li, D. Harabor, P. J. Stuckey, A. Felner, H. Ma, S. Koenig, Disjoint splitting for conflict-based search for multi-agent path finding, in: International Conference on Automated Planning and Scheduling, 2019, pp. 279–283.

[19] J. Li, E. Boyarski, A. Felner, H. Ma, S. Koenig, Improved heuristics for multi-agent path finding with conflict-based search, in: International Joint Conference on Artificial Intelligence, 2019, pp. 442–449.

[20] J. Li, D. Harabor, P. J. Stuckey, H. Ma, G. Gange, S. Koenig, Pairwise symmetry reasoning for multi-agent path finding search, Artificial Intelligence 301 (2021) 103574.

[21] M. A. Erdmann, T. Lozano-Pérez, On multiple moving objects, Algorithmica 2 (1987) 477–521.

[22] H. Ma, D. D. Harabor, P. J. Stuckey, J. Li, S. Koenig, Searching with consistent prioritization for multi-agent path finding, in: AAAI Conference on Artificial Intelligence, 2019, pp. 7643–7650.

[23] J. Švancara, M. Vlk, R. Stern, D. Atzmon, R. Barták, Online multi-agent pathfinding, in: AAAI Conference on Artificial Intelligence, 2019, pp. 7732–7739.

[24] H. Ma, A competitive analysis of online multi-agent path finding, in: International Conference on Automated Planning and Scheduling, 2021, pp. 234–242.

[25] P. Surynek, Multi-goal multi-agent path finding via decoupled and integrated goal vertex ordering, in: AAAI conference on artificial intelligence, 2021, pp. 12409–12417.

[26] H. Zhang, J. Chen, J. Li, B. Williams, S. Koenig, Multi-agent path finding for precedence-constrained goal sequences, in: AAMAS, 2022, pp. 1464–1472.

[27] W. Hönig, S. Kiesel, A. Tinka, J. W. Durham, N. Ayanian, Conflict-based search with optimal task assignment, in: International Conference on Autonomous Agents and Multiagent Systems, 2018, pp. 757–765.

[28] V. Nguyen, P. Obermeier, T. C. Son, T. Schaub, W. Yeoh, Generalized target assignment and path finding using answer set programming, in: International Joint Conference on Artificial Intelligence, 2017, pp. 1216–1223.

[29] C. Henkel, J. Abbenseth, M. Toussaint, An optimal algorithm to solve the combined task allocation and path finding problem, IEEE/RSJ International Conference on Intelligent Robots and Systems (2019).

[30] Z. Ren, S. Rathinam, H. Choset, Cbss: A new approach for multiagent combinatorial path finding, IEEE Transactions on Robotics 39 (2023) 2669–2683.
[31] N. M. Kou, C. Peng, H. Ma, T. K. S. Kumar, S. Koenig, Idle time optimization for target assignment and path finding in sortation centers, in: AAAI Conference on Artificial Intelligence, 2020, pp. 9925–9932.
[32] D. Silver, Cooperative pathfinding, in: AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 2005, pp. 117–122.
[33] F. Grenouilleau, W. van Hoeve, J. N. Hooker, A multi-label A* algorithm for multi-agent pathfinding, in: International Conference on Automated Planning and Scheduling, 2019, pp. 181–185.
[34] H. Ma, Target assignment and path planning for navigation tasks with teams of agents, Ph.D. thesis, University of Southern California, 2020.
[35] Z. Chen, J. Alonso-Mora, X. Bai, D. D. Harabor, P. J. Stuckey, Integrated task assignment and path planning for capacitated multi-agent pickup and delivery, IEEE Robotics and Automation Letters 6 (2021) 5816–5823. doi:10.1109/LRA.2021.3074883.